

Dynamic Homotopy for Ill-Conditioned Power Flow: Reproduction and Improvements

Based on A. Lima-Silva and F. D. Freitas, "Exploring a Dynamic Homotopy Technique to Enhance the Convergence of Classical Power Flow Iterative Solvers in Ill-Conditioned Power System Models", *Energies* 17(18):4642, 2024

Jonayed, Hasbi, Apurbo, Sizan, Sadman
CSE 402 Numerical Analysis Sessional

Power flow as a nonlinear system

- Steady state of a grid: solve $g(x) = 0$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^n$
- Each bus gives a power balance equation

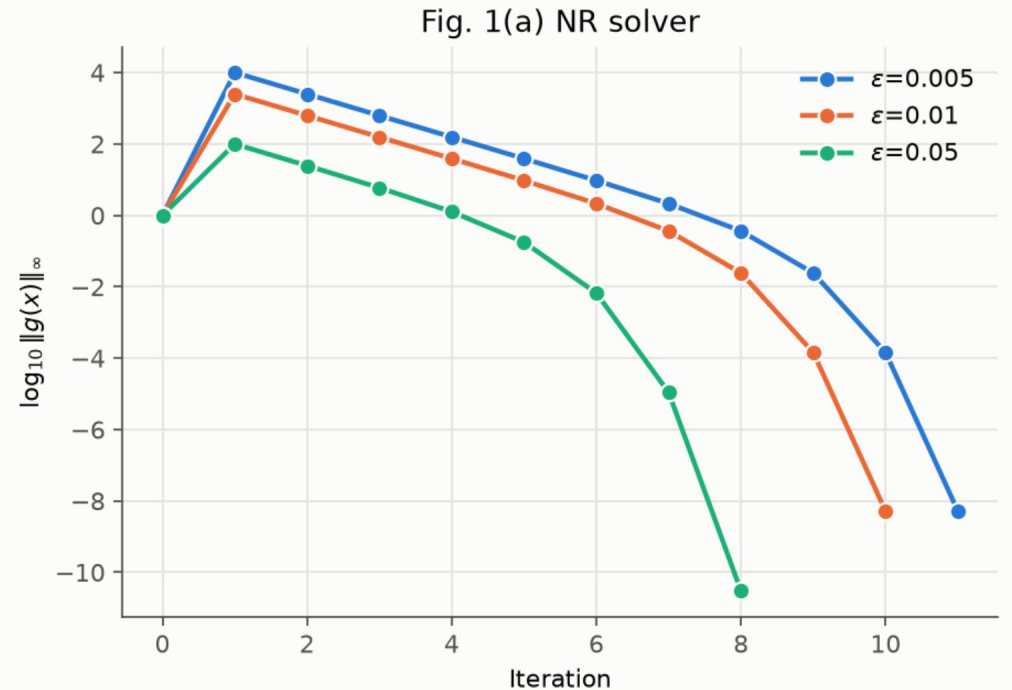
$$\Delta S = V \cdot \text{conj}(Y V) - (\text{generation} - \text{load})$$

- Unknowns x : voltage angles θ (PV and PQ buses) and magnitudes $|V|$ (PQ buses)
- Flat start x_0 : all $\theta = 0$, all $|V| = 1$
- Newton from the flat start diverges on all 5 large grids of the paper

Newton-Raphson and why it fails here

$$x_{i+1} = x_i - J(x_i)^{-1} g(x_i)$$

- One sparse LU of J per iteration
- Stop at $\|g\|_\infty < 10^{-8}$, at most 10 iterations
- Quadratic convergence, but only locally
- At the flat start J is nearly singular: $|\lambda|_{\max} / |\lambda|_{\min}$ up to 1.2×10^8
- Small eigenvalue, huge step, divergence



The fixed-point homotopy

Start from an easy problem with a known root and deform it into g :

$$G(x, t) = t \cdot g(x) + (1 - t) \cdot K (x - x_0), t \in [0, 1]$$

- $t = 0$: $G = K (x - x_0)$, root x_0 (the flat start)
- $t = 1$: $G = g(x)$, the real problem
- In between, the root moves along a smooth path $x(t)$
- K weights the easy part; the paper uses $K = 10^{-4}$

From the path to an ODE

$G(x(t), t) = 0$ along the path. Differentiating with the chain rule:

$$G_x \dot{x} + G_t = 0 \Rightarrow \dot{x} = -G_x^{-1} G_t, x(0) = x_0$$

with

$$G_x = t J(x) + (1 - t) K I, G_t = g(x) - K (x - x_0)$$

- Root finding becomes an initial value problem (Davidenko equation)
- Each evaluation of $\dot{x}$ is one sparse solve with G_x , i.e. one LU factorization
- We count LU factorizations as the cost of every method

Integration schemes used by the paper

With $f(x, t) = -G_x(x, t)^{-1} G_t(x)$:

Scheme	Step	LUs	Order
Forward Euler	$x_{k+1} = x_k + \Delta t f(x_k, t_k)$	1	1
Backward Euler (paper)	$x_{k+1} = x_k + \Delta t f(x_k, t_{k+1})$	1	1
RK2 (Heun)	$x_{k+1} = x_k + (\Delta t/2)(k_1 + k_2)$	2	2

- BE is implicit; the paper does a single fixed-point iteration from x_k
- The only difference from FE is that G_x is evaluated at t_{k+1} instead of t_k

The first step: forward vs backward Euler

At $t = 0$, $x = x_0$ we have $G_t = g(x_0)$. The two schemes differ only in G_x .

Forward Euler

$$G_x(x_0, 0) = K I$$

$$x_1 = x_0 - (\Delta t_0 / K) g(x_0) = x_0 - 50 g(x_0)$$

The network does not appear. Error factor $1 - \lambda/\delta$ per eigenvalue; mismatch grows to 10^{12}

Backward Euler

$$G_x(x_0, \Delta t_0) = \Delta t_0 J + (1 - \Delta t_0) K I$$

$$x_1 = x_0 - (J + \delta I)^{-1} g(x_0), \delta \approx K/\Delta t_0 = 0.02$$

A Newton step with a shifted Jacobian. Error factor $\delta/(\lambda + \delta) < 1$; mismatch 532 to 65

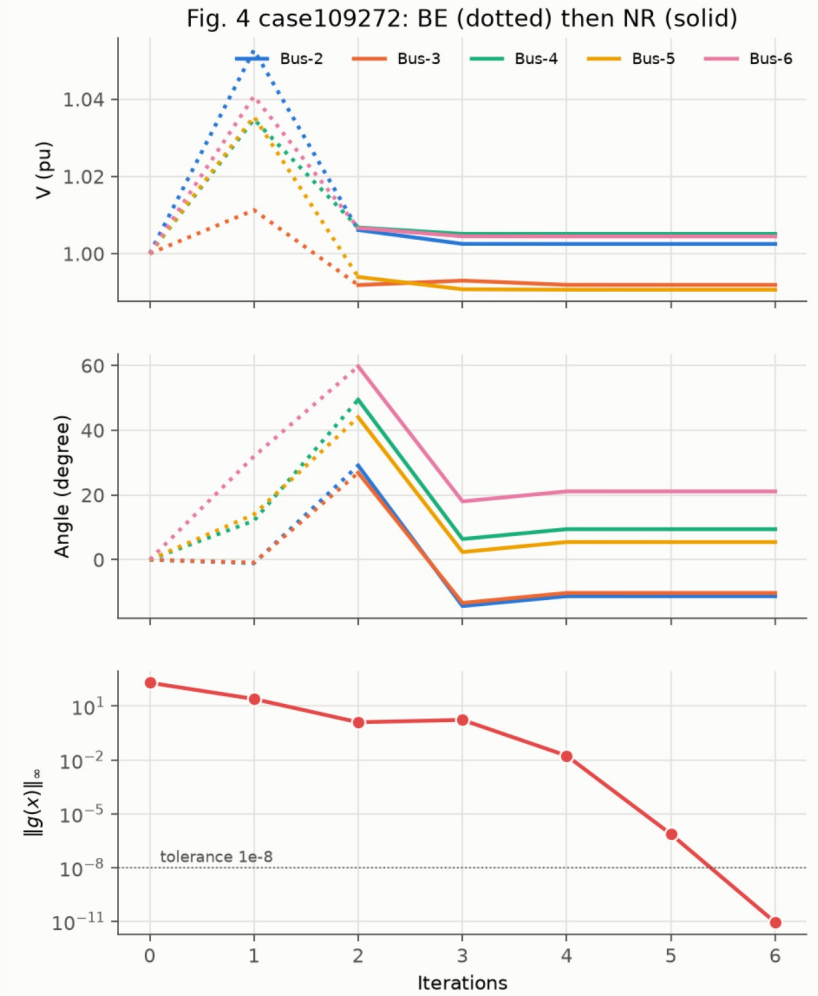
The ODE is stiff at the start, so the implicit scheme is the stable one.

The complete method

1. Start at the flat start x_0 , $t = 0$
2. BE step to $t = 0.005$
3. BE step to $t = 1$
4. Newton-Raphson from $x(1)$

$K = 10^{-4}$, $\Delta t_0 = 0.005$, path $\{0, 0.005, 1\}$

Cost: 2 LUs for the path plus about 4 Newton iterations, where Newton alone diverges. The paper also finishes with FDXB instead of Newton.



Reproduction of the paper's results

Example, Table 4 (case18482, backward Euler): $\|g\|_\infty$ along the path, then Newton

	t = 0	0.005	0.01	0.51	1	NR 1	NR 2	NR 3
Ours	533	65	33	7.2	3.7	0.042	1.5×10^{-5}	9.3×10^{-12}
Paper	532	65	33	7.2	3.7	0.04	1.5×10^{-5}	1.0×10^{-11}

- Tables 2 to 5: 246 of 255 values within a factor of 2 of the paper
- FE and RK2 blow up at the same places as in the paper
- Table 7 iteration counts match on the 5 large grids except in 2 cells
- Figures 4 and 5: same voltage paths and final state

Checking which root is reached

- Power flow equations have several roots
- $\|g(x)\|_\infty < 10^{-8}$ shows that x is a root, not that it is the intended one
- Reference root $V(\text{ref})$: Newton from the initial guess stored in the case file
- A run counts as correct only if

$$\text{max over buses of } |V - V(\text{ref})| < 10^{-4} \text{ pu}$$

- All converged runs in the paper's Table 7 pass
- On our wider test set, 14 of the 141 converged runs of the paper's method reached a different root

Evaluation setup for the improvements

- 34 grids: the paper's 9 plus 25 more from the same Zenodo records
- 5 settings of $(\Delta t_0, K)$, path $\{0, \Delta t_0, 1\}$, giving 170 runs per configuration:

Setting	Δt_0	K
S1 (paper)	0.005	10^{-4}
S2, S3 (paper, Sec. 4.4)	0.05, 0.1	$10^{-3}, 2 \times 10^{-3}$
S4 (weak K), S5 (strong K)	0.005	$10^{-5}, 10^{-3}$

- Solved: Newton converges within 10 iterations and reaches the reference root
- Cost: LU factorizations per run. Each change is a separate on/off switch

Improvement 1: predictor-corrector

- Predictor: the BE step gives a point $\tilde{x}$ near, but not on, the path
- Corrector: one Newton step on $G(x, t_{k+1}) = 0$ at the fixed new time

$$x_{k+1} = \tilde{x} - G_x(\tilde{x}, t_{k+1})^{-1} G(\tilde{x}, t_{k+1})$$

- Standard scheme of continuation methods; one extra LU per step
- Result: 152 of 170 solved (paper: 127)
- Wrong roots drop from 14 to 2; 8.43 LUs per run (paper: 8.04)

Improvement 2: optimal multiplier (Iwamoto)

Damped Newton: take $x + \mu\Delta x$. With $a = g(x)$, $c = g(x + \Delta x)$ and $J\Delta x = -a$:

$$g(x + \mu\Delta x) \approx (1 - \mu) a + \mu^2 c$$

Minimising the squared norm gives a cubic in μ :

$$2(c \cdot c) \mu^3 - 3(a \cdot c) \mu^2 + (a \cdot a + 2 a \cdot c) \mu - a \cdot a = 0$$

- No extra LU; $\mu \rightarrow 1$ near the root, so quadratic convergence is kept
- Alone: 7.14 LUs per run instead of 8.04 (about 11% fewer), 143 solved
- With the corrector: 152 solved at 7.57 LUs

Improvement 3: Richardson step control

Compare one step of size h with two of size $h/2$ (method of order p):

$$\text{err} \approx \|y(h/2) - y(h)\|_{\infty} / (2^p - 1)$$

$$\text{accept if } \text{err} \leq \text{tol}, y \approx y(h/2) + (y(h/2) - y(h)) / (2^p - 1)$$

$$\text{next } h = 0.9 \cdot h \cdot (\text{tol} / \text{err})^{1/(p+1)}$$

- 151 of 170 solved; 156 with the multiplier, the most of any configuration
- Solves some of the hardest cases, at 20 to 30% more LUs
- Under error control BE is still cheapest: 10.6 LUs vs FE 37, RK2 48, RK4 87

Results on the 170 runs

Configuration	Solved	Wrong root	LUs per run
Paper's method	127	14	8.04
Corrector	152	2	8.43
Multiplier	143	12	7.14
Multiplier + corrector	152	3	7.57
Richardson (BE)	151	-	10.6
Multiplier + Richardson	156	8	9.75

- Corrector: fewer wrong roots. Multiplier: lower cost. Richardson: the hardest cases
- Multiplier + corrector solves 25 more runs than the paper at lower cost

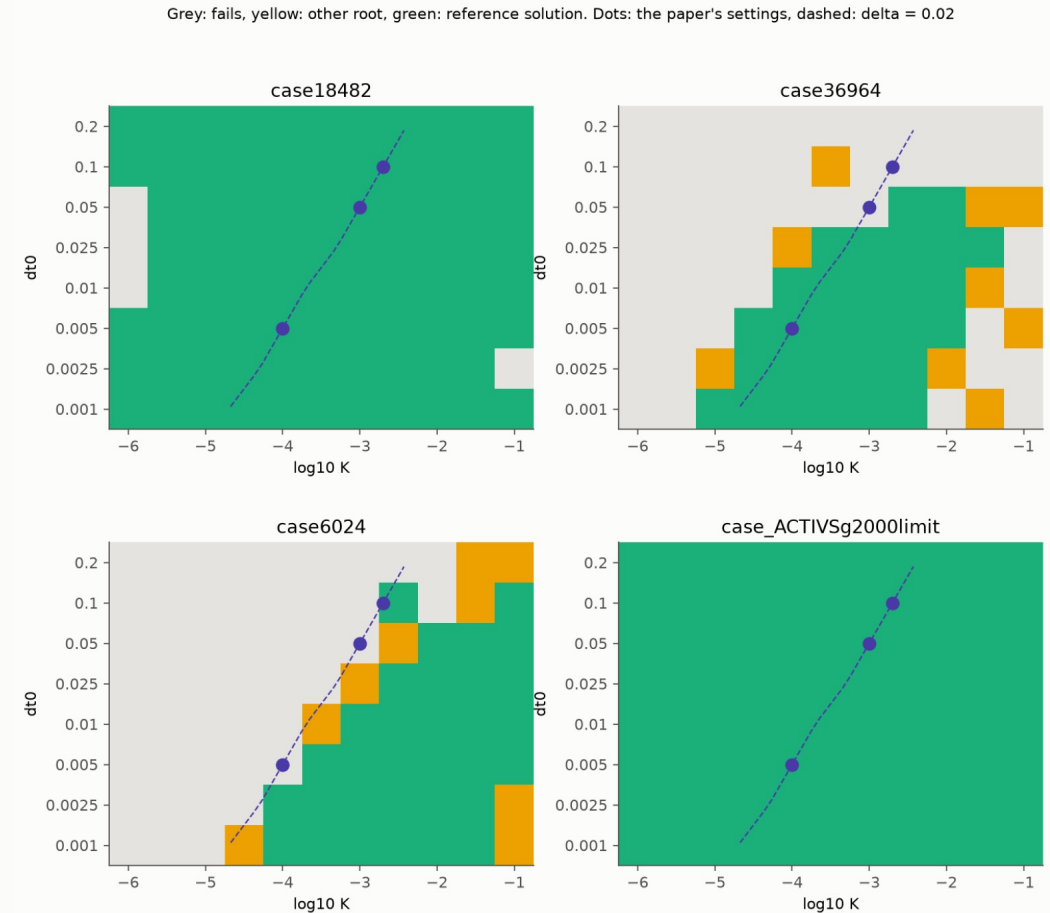
Changes that did not help

Change	Solved	Reason
RK4 with fixed steps	13	Explicit; higher order does not fix stiffness
BE with 3 fixed-point iterations	89	Not a contraction for large Δt , iterates drift
Newton homotopy $g(x) - (1 - t) g(x_0)$	75	$G_x = J$, no K I shift, J stays near-singular
Scaled homotopy $K D (x - x_0)$	129	Almost the same as the paper (127)
Adaptive step halving	132	Small gain; rejected steps still cost LUs

The Newton homotopy result shows that the K I shift is what makes the paper's method work.

Studies from our proposal presentation

- Power method and shifted inverse iteration: $|\lambda|_{\min}$ of J is 4×10^{-4} to 6×10^{-3} ; the shift $\delta = 0.02$ lowers the eigenvalue ratio from 1.2×10^8 to about 2×10^6
- Sweep over $(K, \Delta t_0)$, right: the method works in a band of constant $\delta = K/\Delta t_0$
- Golden-section search for δ : 23 of 25 new grids solved instead of 19, but costs about 21 solves
- Our own Gauss / LU with pivoting: same iterates to 5×10^{-12} , but 424 s vs 0.04 s (no sparsity)



Conclusions

- The paper's first BE step is a Newton step on $J + \delta I$, which is why it works where Newton fails
- We reproduced the method and its tables from scratch in Python
- A converged run is not always the intended root; we check every run against a reference
- Predictor-corrector, optimal multiplier and Richardson step control raise the solved runs from 127 to 152 to 156 of 170
- Backward Euler remains the cheapest integrator, even with error control

Thank you. Questions?